Programovanie v C++: Základné vlastnosti objektovo orientovaného programovania

Prednáška 4: Zapúzdrenie, Dedičnosť a Polymorfizmus

Programovanie mechatronických aplikácií
ÚAMT FEI STU

PMEAP Prednáška 4

Evolúcia programovacích paradigiem

- Klasické "procedurálne" jazyky (ako napríklad C) sa zameriavajú na otázku: *"Čo by mal program urobiť ako ďalšie v poradí?"*.
- Štruktúra takýchto programov spočíva v rozdelení problému na sady úloh a podúloh, vytvorení funkcií a inštruovaní počítača, aby ich vykonal v sekvencii.
- Pri veľkom množstve dát alebo veľkom počte úloh tento prístup rýchlo vedie k neudržateľným a vysoko komplexným systémom.

Motivácia: Modelovanie reality

Príklad: Softvérový model automobilu

V procedurálnom prístupe by program obsahoval stovky nesúvisiacich premenných uchovávajúcich informácie o rôznych súčiastkach a funkcie by do nich voľne pristupovali. Neexistoval by spôsob, ako logicky zoskupiť všetok kód, ktorý sa týka výlučne, povedzme, len kolies.

Ľudia uvažujú o svete prostredníctvom interagujúcich objektov. Hovoríme o interakciách medzi volantom, pedálmi a kolesami. OOP poskytuje programátorom nástroje na "zabalenie" týchto detailov do úhľadných, sebestačných schránok (objektov). Programátor tak môže uvažovať na vyššej úrovni abstrakcie a sústrediť sa len na interakcie medzi nimi.

Tri piliere OOP

Existuje množstvo definícií objektovo-orientovaného programovania, no jeho základ sa opiera o tri primárne vlastnosti:

1. **Zapuzdrenie (Encapsulation):** Zoskupovanie súvisiacich dát a funkcií dohromady do objektov a definovanie prísneho rozhrania (interface) k týmto objektom.
2. **Dedičnosť (Inheritance):** Umožňuje opakované použitie (znovupoužiteľnosť) kódu medzi vzájomne súvisiacimi dátovými typmi.
3. **Polymorfizmus (Polymorphism):** Umožňuje hodnote nadobudnúť niekoľko rôznych typov a určuje **počas behu programu (at runtime)**, ktoré konkrétne funkcie sa majú zavolať na základe jej skutočného typu.

1. Pilier: Zapuzdrenie (Encapsulation)

Zapuzdrenie označuje zabalenie vzájomne súvisiacich vecí k sebe do jedného balíka. V jazyku C++ to robíme pomocou Tried (Classes).

Rozhranie ako čierna skrinka (Black Box)

Ak nám iný inžinier odovzdá triedu na použitie, nepotrebujeme poznať presný vnútorný mechanizmus, ako funguje vo vnútri. Potrebujeme poznať iba jej verejné (public) metódy a dáta – jej rozhranie (interface).

Prax skrývania implementačných detailov pred klientským kódom (tým, ktorý triedu volá) sa nazýva **Data hiding** (skrývanie dát).

Analógia zapuzdrenia: Vedenie vozidla

Koncept rozhrania sa často prirovnáva k prevádzke automobilu.

- Keď šoférujete, absolútne vás nezaujíma, akým mechanizmom presne volant fyzicky zatáča kolesá (hrebeňové riadenie, servo atď.).
- Zaujíma vás výlučne to, že rozhranie (interface), ktoré vám auto poskytuje (samotný volant), vám umožňuje dosiahnuť váš cieľ.

Preto C++ implicitne nastavuje členy triedy ako súkromné (interiérové detaily), čím programátora núti definovať `public` a `private` prístup.

Odosielanie správ medzi objektmi

Objektovo-orientovaný program si možno predstaviť ako definovanie toho, aké objekty existujú a aké informácie každý z nich ovláda. Následne si tieto nezávislé objekty medzi sebou **posielajú správy** (tým, že si vzájomne volajú svoje metódy), aby si vymieňali informácie a inštruovali sa, akú činnosť majú vykonať.

2. Pilier: Dedičnosť (Inheritance)

Dedičnosť nám dovoľuje definovať hierarchie súvisiacich tried.
Predstavme si, že vyvíjame inventarizačný softvér pre vozidlá, vrátane osobných áut a kamiónov.
Namiesto vytvorenia dvoch nesúvisiacich tried, kde by sme museli duplikovať všetok spoločný kód pre vozidlá, použijeme dedičnosť.

- Vytvoríme spoločnú triedu `Vehicle` (Vozidlo), do ktorej umiestnime spoločný kód.
- Následne špecifikujeme triedy `Car` a `Truck`, ktoré tento kód z triedy Vehicle priamo zdieľajú (zdedia).

Príprava rodičovskej triedy (Base Class)

Zadefinujeme našu základnú triedu `Vehicle`.

```
1  class Vehicle {
2  protected:
3      string license;
4      int year;
5
6  public:
7      Vehicle(const string &myLicense, const int myYear)
8          : license(myLicense), year(myYear) {}
9
10     const string getDesc() const {
11         return license + " from " + stringify(year);
12     }
13     const string &getLicense() const { return license; }
14     const int getYear() const { return year; }
15 };
```

Analýza kódu triedy Vehicle

Z inžinierskeho hľadiska si všimnite špecifické konštrukcie:

- Kľúčové slovo `protected` funguje podobne ako `private`, no jeho špecifiká sú kľúčové pre dedičnosť (rozoberieme neskôr).
- **Inicializačný zoznam (Member initializer syntax):** Za konštruktorom nasleduje dvojbodka a čiarkami oddelený zoznam inicializácie napr. `license(myLicense)`.
- Inicializačný zoznam je extrémne dôležitý pre inicializáciu konštantných (`const`) členov objektu ešte pred tým, než sa vôbec spustí samotné telo konštruktora.

Vytvorenie odvodenej triedy (Derived Class)

Teraz špecifikujeme triedu `Car`, ktorá zdedí (**inherit**) všetok kód a vlastnosti z `Vehicle`, ale s vlastnými prídavkami.

```
1 class Car : public Vehicle { // Definuje dedicnost z Vehicle
2     string style;
3
4 public:
5     Car(const string &myLicense, const int myYear, const string &myStyle)
6         : Vehicle(myLicense, myYear), style(myStyle) {}
7
8     const string &getStyle() { return style; }
9 };
```

Trieda `Car` má teraz **všetky** dátové členy a metódy triedy `Vehicle` a k nim navyše vlastný člen `style` a metódu `getStyle`.

Nomenklatúra v hierarchii

Hovoríme, že trieda `Car` dedí z triedy `Vehicle`. To je ekvivalentné terminológii:

- `Car` je odvodená trieda (**derived class**) a `Vehicle` je základná trieda (**base class**).
- V iných jazykoch (napr. Java) sa často používajú termíny podtrieda (**subclass**) a supertrieda (**superclass**).

V diagramoch sa hierarchia kreslí so šípkami smerujúcimi vždy z odvodených tried k ich základnej triede.

Volanie rodičovského konštruktora

Všimnite si kritický riadok v konstruktore `Car`:

```
1    Car( ... )
2        : Vehicle(myLicense, myYear), style(myStyle) {}
```

- Tu využívame inicializačný zoznam na to, aby sme explicitly **zavolali konštruktor základnej triedy (`Vehicle`)**.
- Dôvod: Z hľadiska správy pamäte musíme mať kompletne skonštruovaný a naalokovaný objekt `Vehicle` predtým, než začneme konštruovať dodatočné komponenty, ktoré k nemu pridáva `Car`.
- Ak toto Explicitné volanie vynecháte, kompilátor sa pokúsi vyvolať "defaultný" konštruktor triedy Vehicle bez argumentov.

Relačná paradigma: Is-a vs. Has-a

Pri objektovom návrhu môžeme vzťah závislosti medzi triedou A a triedou B popísať dvoma spôsobmi.

1. **Vzťah "Má"(Has-a):** Každý objekt triedy A vlastní objekt B. Príklad: Každé Vozidlo (Vehicle) má objekt reťazca textu (`license`). Tieto vzťahy by mali byť implementované jednoduchým pridaním polí do triedy, **nie dedičnosťou**.
2. **Vzťah "Je"(Is-a):** Každá inštancia A je priamo inštanciou B. Príklad: Každé Auto (`Car`) **je** Vozidlom (`Vehicle`). Tieto vzťahy sú implementované pomocou dedičnosti. Bolo by obrovskou chybou návrhu, ak by Vehicle dedil zo stringu (lebo vozidlo nie je textový reťazec).

Prekrývanie metód (Overriding Methods)

Môžeme požadovať, aby sa popis pre autá (`Car`) generoval iným spôsobom, ako pre všeobecné vozidlá (`Vehicle`). Môžeme jednoducho **redefinovať** funkciu `getDesc` priamo v triede `Car`.

```
1  class Car : public Vehicle {
2      string style;
3  public:
4      Car(const string &myLicense, const int myYear, const string &myStyle)
5          : Vehicle(myLicense, myYear), style(myStyle) {}
6
7      // Overriding the member function
8      const string getDesc() {
9          return stringify(year) + " " + style + ": " + license;
10     }
11 };
```

Keď teraz zavoláme `getDesc` na objekte `Car`, systém použije túto predefinovanú funkciu. Tento proces sa nazýva **Prekrývanie (Overriding)**.

Princíp: Programming by Difference

Pri definovaní odvodených tried specifikujeme **výhradne len to, v čom sa odlišujú** od svojich základných rodičovských tried.

- Táto extrémne silná technika redukcie redundantného kódu sa nazýva Programovanie podľa rozdielu (**programming by difference**).
- Z technického hľadiska ale môžeme metódy iba prekrývať alebo pridávať nové. **Nikdy nesmieme/nemôžeme odstrániť** (odobrať) funkcionalitu, ktorá už bola prítomná v rodičovskej triede.

Modifikátor protected a Dedičnosť

Prečo sme v `Vehicle` použili `protected` a nie `private`?

- Ak by sme premenné `year` a `license` deklarovali ako `private`, systém by zabránil ich prístupu aj zo samotnej odvodených tried (napr. z `Car`).
- Prístupový modifikátor `protected` rieši presne tento inžiniersky problém: dovoľuje bezpečný prístup do pamäte z odvodených (derived) tried vo vnútri hierarchie, ale zakazuje prístup pre bežný externý kód (správa sa navonok ako `private`).

Limit viditeľnosti pri dedení (Inheritance Access Specifiers)

Kľúčové slovo `public` použité pri špecifikovaní základnej triedy (napr. `class Car : public Vehicle {...}`) definuje limit pre viditeľnosť zdedených metód a členov v odvodenej triede.

- **Public dedičnosť:** Normálne by ste mali na tomto mieste používať `public`. Znamená to, že zdedené metódy deklarované v rodičovi ako verejné (`public`) zostávajú naďalej verejnými (`public`) aj v odvodenej triede.
- **Protected / Private dedičnosť:** Špecifikovanie `protected` pri dedení by spôsobilo, že zdedené metódy (dokonca aj tie, ktoré boli v základnej triede pôvodne `public`) budú mať v odvodenej triede nanajvýš chránenú (`protected`) viditeľnosť.

Kompletná tabuľka modifikátorov dedičnosti

Pre kompletnú inžiniersku tabuľku efektov rôznych modifikátorov prístupu pri dedení navštívte technickú dokumentáciu:

`https://en.wikibooks.org/wiki/C++_Programming/Classes/Inheritance`

3. Pilier: Polymorfizmus (Polymorphism)

Slovo Polymorfizmus znamená "mnoho tvarov". Odvíja sa od schopnosti jedného objektu figurovať pod viacerými pamäťovými typmi.

Koncept zameniteľnosti

Ak máme funkciu, ktorá na vstupe očakáva objekt `Vehicle`, môžeme jej bez akéhokoľvek problému bezpečne podsunúť inštanciu typu `Car`, pretože v OOP **každé Auto je Vozidlo**.

Tento prístup plne a identicky platí pre referencie aj pre pamäťové smerníky. Všade, kde kompilátor žiada `Vehicle *`, môžeme bezpečne zapísať `Car *`.

Architektonický problém pri neskorej väzbe

Tu však narážame na kritický technický problém, ak sa metóda priamo nešpecifikuje správne.

```
1 Car c("VANITY", 2003);
2 Vehicle *vPtr = &c;
3 cout << vPtr->getDesc();
```

(Zápis `vPtr->getDesc()` slúži na súčasné dereferencovanie smerníka a zavolanie člena, je ekvivalentom `(*vPtr).getDesc()`).
Keďže kompilátor vníma (a má deklarovaný) `vPtr` staticky ako ukazovateľ na typ `Vehicle`, **vyvolá sa verzia metódy** `getDesc` **z triedy Vehicle**. A to aj napriek tomu, že na danej adrese fyzicky leží objekt typu `Car`.

Riešenie: Virtuálne funkcie (virtual)

Zvyčajne si želáme, aby systém priamo **počas behu (at runtime)** analyzoval, na aký typ objektu pamäťový smerník naozaj fyzicky ukazuje a zavolal korektnú predefinovanú funkciu.
Tento inžiniersky posun nazývame neskorá väzba (dynamic dispatch). Docielime to pridaním kľúčového slova `virtual` pred deklaráciu metódy v základnej triede.

```
1 class Vehicle {
2     // ...
3     virtual const string getDesc() { ... }
4 };
```

Po tejto zmene náš predchádzajúci kód úspešne, správne a bezpečne zavolá verziu z triedy `Car`.

Polymorfizmus a Referencie

Pretože referencie v jazyku C++ pod kapotou implicitne využívajú mechanizmus smerníkov (pointers), presne rovnaké problémy a ich riešenia platia aj pre referencie.

```
1 Car c("VANITY", 2003);
2 Vehicle &v = c;
3 cout << v.getDesc();
```

Tento kód na adrese referencie v korektne nájde a zavolá Car-verziu metódy `getDesc` len za striktného predpokladu, že sme v základnej triede definovali túto metódu s modifikátorom `virtual`.

Dedičnosť parametra virtual

Ak raz metódu v základnej triede C vyhlásite ako `virtual`, stáva sa kompilátorom braná ako virtuálna **vo všetkých** od nej odvodených triedach (subclasses).
Stáva sa tak bez ohľadu na to, či pri je prekrytí v podtriedach explicitne toto kľúčové slovo napíšete. Z dôvodu čitateľnosti a udržateľnosti (clarity) kódu pre iných inžinierov sa to však dôrazne odporúča vždy zapísať znova.

Prechod k Abstrakcii: Pure virtual functions

Niekedy nemá zmysel metódu v základnej triede vôbec implementovať.

- Neexistuje rozumný spôsob, ako by sa mala generovať návratová hodnota `getDesc` pre úplne abstraktné "všeobecné Vozidlo" (lebo v realite "všeobecné vozidlo", ktoré nie je ani autom ani kamiónom, de-facto neexistuje).
- Napriek tomu chceme silou architektúry na kompilátore **vynútiť (require)**, aby každá z nej odvodená podtrieda túto funkciu povinne vlastnila a definovala si ju.

Implementácia abstraktnej metódy

V C++ vynecháme telo funkcie z `Vehicle` tým, že ju označíme ako rýdzo virtuálnu (Pure virtual) pomocou veľmi špecifickej syntaxe `= 0`.

```
1 class Vehicle {
2     // ...
3     // Rydzo virtualna funkcia (Pure virtual)
4     virtual const string getDesc() = 0;
5 };
```

Číslica `= 0` komunikuje C++ linkéru, že k tejto funkcii z našej strany nebude nikdy dodaná žiadna implementačná definícia.

Abstraktné triedy (Abstract Classes)

Tento zásah (= 0) má fundamentálny vplyv na samotnú triedu.

- Kompilátor vám po novom **absolútne zakáže** vytvoriť v pamäti inštanciu samotnej triedy `Vehicle`.
- Vytvoriť môžete už len inštancie jej dcérskych tried (napr. `Car` alebo `Truck`), ktoré tento vynútený sľub rozhrania plnia a funkciu zadefinovali.

Trieda, ktorá obsahuje aspoň jednu *pure virtual* funkciu sa stáva tzv. **abstraktnou triedou** (abstract class). Definuje len rozhranie a sľub (interface), ale sama ho neimplementuje, a preto sa z nej nedá konštruovať fyzický objekt v pamäti.

Viacnásobná dedičnosť (Multiple Inheritance)

Na rozdiel od iných bežných objektovo-orientovaných jazykov (ktoré ju neumožňujú), jazyk C++ technicky umožňuje triede **viacero základných rodičovských tried** (multiple base classes) naraz.

```
1 // Auto dedi z dvoch nezavislych rodicov
2 class Car : public Vehicle, public InsuredItem {
3     // ...
4 };
```

Tento zápis prikazuje kompilátoru, aby trieda `Car` obsahovala a zdedila plnú množinu dátových a funkčných členov z oboch tried - tak z triedy `Vehicle`, ako aj z `InsuredItem`.

Problémy viacnásobnej dedičnosti

Z inžinierskeho pohľadu je však viacnásobná dedičnosť extrémne komplikovaná, zákerná a potenciálne nebezpečná (tricky and dangerous).

1. **Kolízia názvov (Ambiguation):** Ak aj `Vehicle` aj `InsuredItem` definujú členskú premennú `x`, nastáva kolízia v pamäti. Pri použití musíte manuálne"disambiguovať" prístup zápisom `Vehicle::x` alebo `InsuredItem::x`.
2. **Dreaded Diamond (Závan Diamantu):** Ak by obaja rodičia zhodou okolností dedili od jedného tretieho, rovnakého predka, v triede `Car` by ste vytvorili dve redundantné, klonované kópie tohto predka (v C++ je riešiteľné tzv. "virtuálnou dedičnosťou", ale vedie k zložitému chaosu).

Odporúčanie: Vo všeobecnosti sa striktne vyhýbajte používaniu viacnásobnej dedičnosti, pokiaľ si absolútne a exaktne nie ste vedomí, čo robíte v RAM a prečo to robíte.

Q & A

Ďakujem za pozornosť!